Backend Architecture & System Documentation

```
## Project: FurniMind AI (ASP.NET Core RESTful API)

This documentation provides an in-depth technical analysis and comprehensive
onboarding guide for the enterprise-grade backend architecture of
**FurniMind AI**. This document is designed for developers, engineering
leads, and stakeholders to understand the system design, data flows, API
contracts, and development standards.

---

## 1. Project Overview

### Project Purpose & Business Context
**FurniMind AI** is an enterprise-grade e-commerce backend infrastructure
powering a premium furniture marketplace integrated with advanced spatial
artificial intelligence. The backend serves as the single source of truth
for:

1. **Premium Marketplace Operations**: Multi-vendor product catalog
management, inventory control, and seamless checkout workflows.
2. **Spatial AI Integration**: Backend coordination for room analysis, 3D
coordinate placement suggestions (`x`, `y`, `z` coordinates with `pitch`,
`yaw`, `roll` rotations).
3. **Multi-Provider Payment Processing**: Integrated payment gateway
orchestration supporting Stripe, PayPal, and Paymob.
4. **User Management & Authentication**: ASP.NET Core Identity with JWT
token-based authentication and role-based authorization (RBAC).
5. **Order & Inventory Tracking**: Complete order lifecycle management,
shipping status tracking, and notification coordination.
6. **Review & Rating System**: Aggregate product ratings and user-generated
reviews with moderation capabilities.

### Backend Responsibilities
The backend is responsible for:
- **API Design**: RESTful API endpoints adhering to HTTP standards and REST
principles.
- **Data Persistence**: SQL Server database with Entity Framework Core ORM
for data access.
- **Authentication & Authorization**: JWT-based stateless authentication
```

```
with role-based access control.
- **Business Logic**: Service layer encapsulation of core domain logic.
- **Data Validation**: Comprehensive input validation at multiple layers
(DTO, Domain, Repository).
- **Error Handling**: Global exception handling with structured error
responses.
- **Notifications**: Email and WhatsApp integration for transactional
notifications.
- **Performance Optimization**: Async/await programming, pagination, query
optimization.

### Core Features
- ✅ **User Authentication & Authorization** - Secure JWT-based auth with
role management
- ✅ **Product Catalog Management** - Multi-vendor product listings with
filtering and search
- ✅ **Shopping Cart & Checkout** - Complete cart management and order
placement workflows
- ✅ **Payment Processing** - Unified interface for multiple payment
providers
- ✅ **Order Management** - Order tracking, status updates, and
cancellation support
- ✅ **Review System** - User reviews and product ratings aggregation
- ✅ **Favorites/Wishlist** - User-specific product favorites management
- ✅ **Multi-channel Notifications** - Email and WhatsApp order
notifications
- ✅ **Category Management** - Hierarchical product categorization
- ✅ **Address Management** - User shipping and billing address management

### Technology Stack
| Layer | Technology | Purpose |
| :--- | :--- | :--- |
| **Framework** | **ASP.NET Core 8.0** | Modern cloud-native web framework |
| **ORM** | **Entity Framework Core 10.0** | Data access and query
generation |
| **Database** | **SQL Server 2019+** | Relational database engine |
| **Authentication** | **ASP.NET Core Identity** | User management and
authentication |
| **Authorization** | **JWT (HS256)** | Stateless token-based authorization
|
| **API Documentation** | **OpenAPI (Swagger)** | Interactive API
specification |
| **Mapping** | **AutoMapper** | DTO/Entity transformation |
```

```
| **Validation** | **Data Annotations** | Attribute-based validation |
| **Async Programming** | **Task-based Asynchrony** | Async/await patterns |
| **Notification** | **SendGrid / WhatsApp API** | Transactional messaging |
| **Dependency Injection** | **Native .NET DI Container** | Service
lifecycle management |

---

## 2. Backend Architecture

### Architectural Style: Onion Architecture (Layered)

The backend adheres to **Onion Architecture** principles, also known as
**Hexagonal Architecture** or **Clean Architecture**. This design ensures
separation of concerns, testability, maintainability, and independence from
external frameworks.

```mermaid
graph TB
    subgraph "Presentation Layer"
        API["🌐 REST API Controllers<br/>Graduation-API"]
    end

    subgraph "Application Layer"
        SERVICE["📦 Business Services<br/>DTOs, Interfaces, Mappers"]
    end

    subgraph "Domain Layer"
        DOMAIN["🏛️ Core Domain Entities<br/>Business Rules, Base Classes"]
    end

    subgraph "Infrastructure Layer"
        DB["🗄️ Data Access<br/>DbContext, Repositories<br/>Entity Framework
Core"]
        EXT["🔌 External Services<br/>Email, WhatsApp, JWT"]
    end

    API -->|Depends On| SERVICE
    SERVICE -->|Depends On| DOMAIN
    SERVICE -->|Depends On| DB
    SERVICE -->|Depends On| EXT
    DB -->|Uses| DOMAIN
    EXT -->|Supports| SERVICE
```
```

```
    style API fill:#4A90E2,color:#fff
    style SERVICE fill:#50C878,color:#fff
    style DOMAIN fill:#FFB347,color:#fff
    style DB fill:#FF6B6B,color:#fff
    style EXT fill:#9B59B6,color:#fff
```

Layered Architecture Breakdown

Layer	Project	Responsibility
Presentation	`Graduation-API`	HTTP request handling, routing, controller logic, request validation
Application	`Graduation-Application`	Business logic, DTOs, Service interfaces, AutoMapper configurations
Domain	`Graduation-Domain`	Core entities, base classes, domain models, business rules
Infrastructure	`Graduation-Infrastructure`	DbContext, repositories, Entity Framework migrations, external service implementations

Dependency Flow

- **Presentation** → Application → Domain
- **Application** → Domain + Infrastructure
- **Infrastructure** → Domain
- **No circular dependencies** - each layer depends only on layers below it

---

3. Full Project Structure

Code

```
Graduation-Solution/
├── Graduation-API/                          # Presentation Layer
│   ├── Controllers/
│   │   ├── AuthController.cs                # Authentication endpoints
(Register, Login, OTP, Reset)
│   │   ├── ProductsController.cs            # Product CRUD and filtering
│   │   ├── CategoriesController.cs          # Category management
│   │   ├── CartController.cs                # Shopping cart operations
│   │   ├── OrderController.cs               # Order creation and tracking
│   │   ├── ReviewsController.cs             # Product reviews and ratings
```

```
│   │   ├── FavoritesController.cs          # Wishlist management
│   │   ├── ProfileController.cs            # User profile operations
│   │   ├── RolesController.cs              # Role management (Admin)
│   │   └── HomeController.cs               # Home/Dashboard routes
│   ├── Program.cs                          # Application startup
configuration
│   └── appsettings.json                    # Environment configuration
│
├── Graduation-Application/                 # Application Layer
│   ├── IServices/                          # Service Interfaces
│   │   ├── IAuthService.cs
│   │   ├── IProductService.cs
│   │   ├── ICartService.cs
│   │   ├── IOrderService.cs
│   │   ├── IReviewService.cs
│   │   ├── IFavoriteService.cs
│   │   ├── ICategoryService.cs
│   │   ├── IProfileService.cs
│   │   ├── IRoleService.cs
│   │   ├── IJwtTokenGenerator.cs
│   │   ├── IEmailService.cs
│   │   ├── IWhatsAppService.cs
│   │   ├── INotificationService.cs
│   │   ├── IGenaricRepositories.cs
│   │   └── IProfileRepository.cs
│   ├── Services/                           # Service Implementations
│   │   ├── AuthService.cs                  # Authentication logic
│   │   ├── ProductService.cs               # Product operations
│   │   ├── CartService.cs                  # Cart management
│   │   ├── OrderService.cs                 # Order processing
│   │   ├── ReviewService.cs                # Reviews and ratings
│   │   ├── FavoriteService.cs              # Favorites operations
│   │   ├── CategoryService.cs              # Category operations
│   │   ├── ProfileService.cs               # User profile management
│   │   ├── RoleService.cs                  # Role assignment and
management
│   │   ├── JwtTokenGenerator.cs            # JWT token creation
│   │   ├── EmailService.cs                 # Email notifications
│   │   ├── WhatsAppService.cs              # WhatsApp notifications
│   │   └── NotificationService.cs          # Notification coordination
│   ├── DTOs/                               # Data Transfer Objects
│   │   ├── UserDTO/
│   │   │   ├── RegisterDto.cs
```

```
│   │   │   ├── LoginDto.cs
│   │   │   ├── ForgotPasswordDto.cs
│   │   │   ├── VerifyOtpDto.cs
│   │   │   ├── ResetPasswordDto.cs
│   │   │   ├── UpdateProfileDto.cs
│   │   │   ├── ChangePasswordDto.cs
│   │   │   └── AuthResponseDto.cs
│   │   ├── ProductDTO/
│   │   │   ├── ProductDto.cs
│   │   │   ├── ProductDetailsDto.cs
│   │   │   ├── CreateProductDto.cs
│   │   │   ├── UpdateProductDto.cs
│   │   │   ├── ProductResponseDto.cs
│   │   │   ├── ProductFilterDto.cs
│   │   │   └── PaginatedResult.cs
│   │   ├── CartDTO/
│   │   │   ├── CartDto.cs
│   │   │   ├── CartItemDto.cs
│   │   │   ├── AddToCartDto.cs
│   │   │   └── UpdateCartItemDto.cs
│   │   ├── OrderDTO/
│   │   │   ├── OrderResponseDto.cs
│   │   │   ├── CreateOrderDto.cs
│   │   │   └── OrderItemDto.cs
│   │   ├── ReviewDTO/
│   │   │   ├── ReviewDto.cs
│   │   │   ├── CreateReviewDto.cs
│   │   │   ├── ReviewDetailsDto.cs
│   │   │   └── AverageRatingDto.cs
│   │   ├── CategoryDTO/
│   │   │   ├── CategoryResponseDto.cs
│   │   │   ├── CreateCategoryDto.cs
│   │   │   └── UpdateCategoryDto.cs
│   │   ├── FavoriteDTO/
│   │   │   └── FavoriteDto.cs
│   │   ├── RolesDTO/
│   │   │   ├── RoleResponseDto.cs
│   │   │   ├── CreateRoleDto.cs
│   │   │   └── AssignRoleDto.cs
│   │   └── Common/
│   │       └── PaginatedResult.cs
│   ├── Mapper/                                # AutoMapper Profiles
│   │   ├── AuthResponseMappingConfig.cs
```

```
│   │   ├── ProductMappingConfig.cs
│   │   ├── ReviewMappingConfig.cs
│   │   ├── CategoryMappingConfig.cs
│   │   ├── UserProfileMappingConfig.cs
│   │   └── RegisterMappingConfig.cs
│   ├── ExternalServices/
│   │   └── EmailServices/
│   │       └── IEmailService.cs
│   ├── Options/
│   │   └── WhatsAppNotificationSettings.cs
│   └── IRepositories/
│       ├── IGenaricRepositories.cs
│       └── IProfileRepository.cs
│
├── Graduation-Domain/                          # Domain Layer
│   ├── Entities/
│   │   ├── BaseEntity.cs                       # Abstract base class (Id,
Timestamps, IsActive)
│   │   ├── ApplicationUser.cs                  # Extended Identity user entity
│   │   ├── Product.cs                          # Product aggregate root
│   │   ├── ProductImage.cs                     # Product image association
│   │   ├── Category.cs                         # Product category
│   │   ├── Workshop.cs                         # Vendor/Workshop entity
│   │   ├── Cart.cs                             # Shopping cart
│   │   ├── CartItem.cs                         # Cart line items
│   │   ├── Order.cs                            # Order aggregate root
│   │   ├── OrderItem.cs                        # Order line items
│   │   ├── OrderStatusHistory.cs               # Order status audit trail
│   │   ├── Review.cs                           # Product reviews
│   │   ├── Favorite.cs                         # User favorites
│   │   ├── Address.cs                          # User addresses
│   │   ├── Discount.cs                         # Discount/Promo codes
│   │   ├── Notification.cs                     # User notifications
│   │   └── FinalResultImage.cs                 # AI spatial analysis results
│   └── Constants/ (if any)
│
└── Graduation-Infrastructure/                  # Infrastructure Layer
    ├── AppDbContext/
    │   ├── ApplicationDbContext.cs              # Entity Framework DbContext
    │   └── ApplicationDbContextFactory.cs       # Design-time factory for
migrations
    ├── Repositories/
    │   ├── GenaricRepositories.cs              # Generic repository
```

```
implementation
    │   ├── ProfileRepository.cs              # User profile repository
    │   └── (Other specialized repositories)
    ├── ProgramService/
    │   └── ServicesAPI/
    │       └── ServiceAPI.cs                 # Service registration
extension method
    ├── Migrations/
    │   ├── [Migration_Timestamp]_[Name].cs  # EF Core migrations
    │   └── ApplicationDbContextModelSnapshot.cs
    ├── Identity/
    │   └── (Identity configuration files)
    └── Database/
        └── ApplicationDbSeeder.cs            # Initial data seeding
```

Folder Responsibilities

Folder	Purpose
Controllers/	HTTP request routing, parameter validation, response formatting
Services/	Business logic implementation, service coordination, data transformation
DTOs/	Data contracts between API and consumers, decoupling domain from API
Entities/	Domain model representations, database schema definitions
Repositories/	Data access abstraction, CRUD operations, query building
Mapper/	AutoMapper profile configurations for entity ↔ DTO transformations
AppDbContext/	Entity Framework configuration, relationship mapping, migrations
Migrations/	Database schema version control and evolution

---

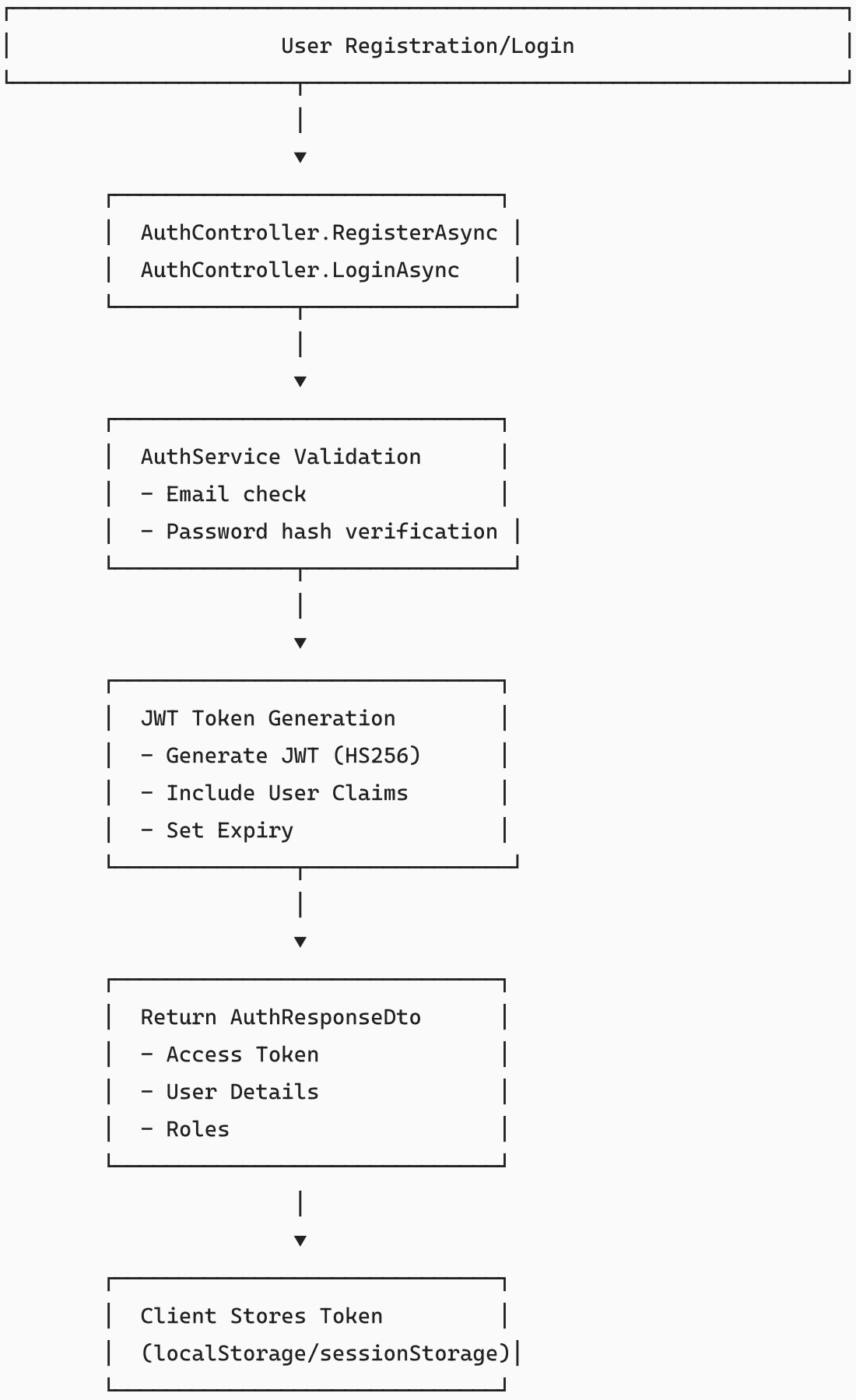
4. Core Systems Documentation

4.1 Authentication & Authorization System

The backend implements **JWT-based stateless authentication** with ASP.NET Core Identity and role-based authorization.

Authentication Flow

Code



JWT Token Structure

C#

```
// JWT Claims included in token
new ClaimsIdentity(new[]
{
    new Claim(ClaimTypes.NameIdentifier, user.Id),           // sub: unique
user ID
    new Claim(ClaimTypes.Name, user.UserName),                //
preferred_username
    new Claim(ClaimTypes.Email, user.Email),                  // email
    new Claim(ClaimTypes.GivenName, user.FullName),           // given_name
    // Roles added as separate claims
    roles.ForEach(role =>
        claims.Add(new Claim(ClaimTypes.Role, role))          // role
    )
})
```

Role-Based Authorization

C#

```
// Example: Admin-only endpoint
[Authorize(Roles = "Admin")]
[HttpPut("{id}")]
public async Task<IActionResult> UpdateCategory(int id, [FromBody]
UpdateCategoryDto dto)
{
    // Only users with "Admin" role can access
    var result = await _categoryService.UpdateCategoryAsync(id, dto);
    return Ok(result);
}

// Example: Protected user endpoint
[Authorize]
[HttpGet]
public async Task<IActionResult> GetCart()
{
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var cart = await _cartService.GetCartAsync(userId);
    return Ok(cart);
}
```

Key Authentication Components

Component	Location	Responsibility
AuthController	`Graduation-API/Controllers/`	Handles register, login, password reset, OTP verification
AuthService	`Graduation-Application/Services/`	Core auth business logic, token generation coordination
JwtTokenGenerator	`Graduation-Application/Services/`	JWT token creation with claims and expiry
ServiceAPI.cs	`Graduation-Infrastructure/`	JWT scheme configuration, Identity setup
ApplicationUser	`Graduation-Domain/Entities/`	Extended Identity user with custom properties

---

4.2 Database Architecture & Entity Relationships

Core Entities & Relationships

Mermaid

```
erDiagram
    APPLICATIONUSER ||--o{ ORDER : places
    APPLICATIONUSER ||--o{ CART : owns
    APPLICATIONUSER ||--o{ REVIEW : writes
    APPLICATIONUSER ||--o{ FAVORITE : has
    APPLICATIONUSER ||--o{ ADDRESS : has

    WORKSHOP ||--o{ PRODUCT : sells
    WORKSHOP ||--o{ REVIEW : receives

    CATEGORY ||--o{ PRODUCT : categorizes

    PRODUCT ||--o{ PRODUCTIMAGE : has
    PRODUCT ||--o{ CARTITEM : added_to
    PRODUCT ||--o{ ORDERITEM : ordered_as
    PRODUCT ||--o{ REVIEW : reviewed_in
    PRODUCT ||--o{ FAVORITE : liked_by

    CART ||--o{ CARTITEM : contains

    ORDER ||--o{ ORDERITEM : contains
    ORDER ||--o{ ORDERSTATUSHISTORY : tracks_status
```

```
    DISCOUNT }o--|| ORDER : applied_to
```

Database Schema Highlights

Base Entity (Inherited by all domain entities):

C#

```
public abstract class BaseEntity<TKey>
{
    public TKey Id { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? UpdatedAt { get; set; }
    public string? CreatedBy { get; set; }
    public string? UpdatedBy { get; set; }
    public bool IsActive { get; set; } = true;
}
```

DbContext Configuration:

C#

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
{
    public DbSet<Workshop> Workshops { get; set; }
    public DbSet<Category> Categories { get; set; }
    public DbSet<Product> Products { get; set; }
    public DbSet<ProductImage> ProductImages { get; set; }
    public DbSet<Address> Addresses { get; set; }
    public DbSet<Favorite> Favorites { get; set; }
    public DbSet<Cart> Carts { get; set; }
    public DbSet<CartItem> CartItems { get; set; }
    public DbSet<Review> Reviews { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<OrderItem> OrderItems { get; set; }
    public DbSet<OrderStatusHistory> OrderStatusHistory { get; set; }
    public DbSet<FinalResultImage> FinalResultImages { get; set; }
    public DbSet<Discount> Discounts { get; set; }
    public DbSet<Notification> Notifications { get; set; }
}
```

Key Relationships:

- **Product** → **Workshop**: One-to-Many with `OnDelete(DeleteBehavior.Restrict)`
- **Product** → **Category**: One-to-Many
- **CartItem** → **Cart**: One-to-Many
- **OrderItem** → **Order**: One-to-Many
- **Review** → **Product**: One-to-Many with cascade delete
- **Review** → **User**: One-to-Many with cascade delete
- **OrderStatusHistory** → **Order**: One-to-Many

Repository Pattern

Generic Repository:

C#

```
public interface IGenaricRepositories<T> where T : class
{
    Task<T> AddAsync(T entity);
    Task<IEnumerable<T>> GetAllAsync();
    IQueryable<T> GetQueryable();
    IQueryable<T> Where(Expression<Func<T, bool>> predicate);
    Task<T?> FirstOrDefaultAsync(Expression<Func<T, bool>> predicate);
    Task UpdateAsync(T entity);
    Task DeleteAsync(T entity);
    Task SaveChangesAsync();
    IQueryable<T> Include(Expression<Func<T, IEnumerable<object>>>
navigationPropertyPath);
}
```

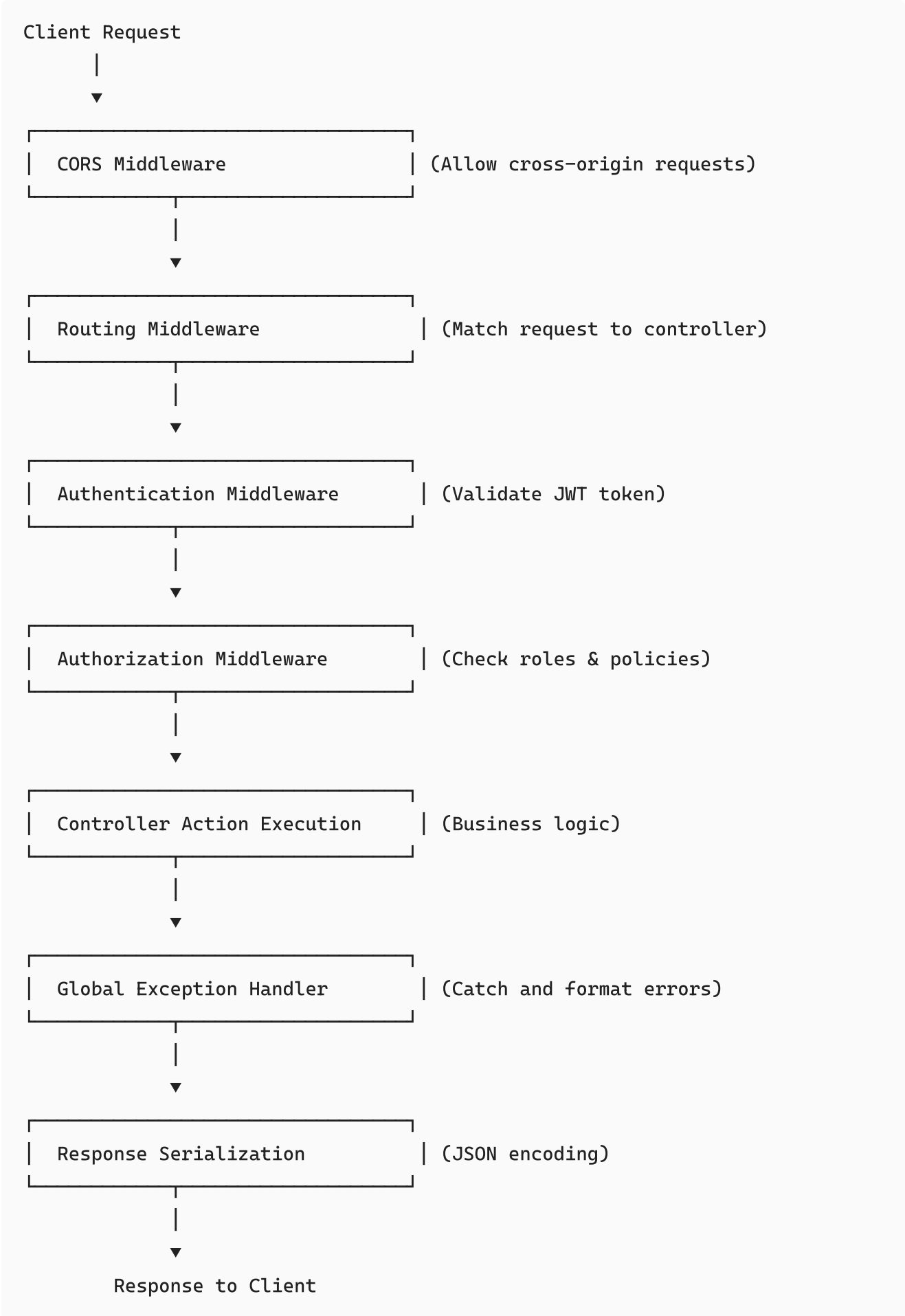
The generic repository pattern provides:

- LINQ query building (`Where` , `Include` , `FirstOrDefaultAsync`)
- CRUD operations (`AddAsync` , `UpdateAsync` , `DeleteAsync`)
- Change tracking management (`SaveChangesAsync`)
- Flexibility for specialized repositories (e.g., `ProfileRepository`)

4.3 Middleware Pipeline & Exception Handling

Request Pipeline

Code



Program.cs Configuration

C#

```
// Middleware Registration Order (in Program.cs)
app.UseRouting();
app.UseCors("AllowAll");                    // CORS policy
app.UseAuthentication();                    // JWT validation
app.UseAuthorization();                     // Role checking
app.MapControllers();                       // Route mapping
app.Run();
```

CORS Configuration

C#

```
services.AddCors(options =>
{
    options.AddPolicy("AllowAll",
        builder =>
        {
            builder
                .AllowAnyOrigin()
                .AllowAnyMethod()
                .AllowAnyHeader();
        });
});
```

Global Exception Handling

All controllers implement consistent error handling:

C#

```
[HttpPost]
public async Task<IActionResult> CreateOrder([FromBody] CreateOrderDto
request)
{
    try
    {
        var order = await _orderService.CreateOrderAsync(userId, request,
User);
        return Ok(new { Message = "Order created successfully", Data = order
});
    }
    catch (ArgumentException ex)
    {
```

```
        return BadRequest(new { Success = false, Message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return Conflict(new { Success = false, Message = ex.Message });
    }
    catch (Exception ex)
    {
        return StatusCode(500, new { Success = false, Message = ex.Message
});
    }
}
```

4.4 Data Validation System

DTO-Level Validation (Data Annotations)

C#

```
public class LoginDto
{
    [Required]
    [EmailAddress]
    public string Email { get; set; }

    [Required]
    [MinLength(6)]
    public string Password { get; set; }
}

public class CreateProductDto
{
    [Required]
    public int CategoryId { get; set; }

    [Required]
    [StringLength(200)]
    public string NameAr { get; set; }

    [Required]
    [StringLength(200)]
```

```
    public string NameEn { get; set; }

    [Range(0.01, double.MaxValue)]
    public decimal Price { get; set; }
}
```

Service-Level Validation

C#

```
public async Task<OrderResponseDto> CreateOrderAsync(string userId,
CreateOrderDto request)
{
    // Business logic validation
    if (string.IsNullOrWhiteSpace(userId))
        throw new ArgumentException("User ID is required");

    var cart = await _cartRepository.FirstOrDefaultAsync(c => c.UserId ==
userId);
    if (cart == null || !cart.Items.Any())
        throw new InvalidOperationException("Cart is empty");

    // Process order...
}
```

Review Rating Validation

C#

```
[HttpPost]
[Authorize]
public async Task<IActionResult> CreateReview([FromBody] CreateReviewDto
dto)
{
    if (dto.Rating < 1 || dto.Rating > 5)
        return BadRequest(new { message = "Rating must be between 1 and 5"
});

    var result = await _reviewService.CreateReviewAsync(userId, dto);
    return Ok(result);
}
```

4.5 Dependency Injection Container

Service Registration (ServiceAPI.cs)

C#

```
public static void AddInfrastructureAPI(
    this IServiceCollection services,
    IConfiguration configuration)
{
    // Database
    services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(configuration.GetConnectionString("GraduationDbOnline")
));

    // Identity
    services.AddIdentity<ApplicationUser, IdentityRole>()
        .AddEntityFrameworkStores<ApplicationDbContext>()
        .AddDefaultTokenProviders();

    // Services Registration
    services.AddScoped<IAuthService, AuthService>();
    services.AddScoped<IProductService, ProductService>();
    services.AddScoped<ICartService, CartService>();
    services.AddScoped<IOrderService, OrderService>();
    services.AddScoped<IReviewService, ReviewService>();
    services.AddScoped<IFavoriteService, FavoriteService>();
    services.AddScoped<ICategoryService, CategoryService>();
    services.AddScoped<IProfileService, ProfileService>();
    services.AddScoped<IJwtTokenGenerator, JwtTokenGenerator>();

    // External Services
    services.AddScoped<IEmailService, EmailService>();
    services.AddScoped<IWhatsAppService, WhatsAppService>();
    services.AddScoped<INotificationService, NotificationService>();

    // Repositories
    services.AddScoped(typeof(IGenaricRepositories<>),
typeof(GenaricRepositories<>));
    services.AddScoped<IProfileRepository, ProfileRepository>();

    // AutoMapper Profiles
```

```
    RegisterMappingConfig.RegisterMappings();
    AuthResponseMappingConfig.Response();
    CategoryMappingConfig.RegisterMappings();
    ProductMappingConfig.RegisterMappings();
    ReviewMappingConfig.RegisterMappings();
}
```

Service Lifetimes

Lifetime	Usage	Example
Transient	New instance per request	Stateless utilities
Scoped	One instance per HTTP request	DbContext, Services, Repositories
Singleton	One instance for application lifetime	Configuration, Cache (if used)

Note: All services use `AddScoped` to ensure DbContext isolation per request.

---

5. Features & Modules Documentation

5.1 Authentication Module

Endpoints:

Method	Route	Description	Auth
POST	`/api/auth/register`	User registration	❌
POST	`/api/auth/login`	User login	❌
POST	`/api/auth/forgot-password`	Request password reset OTP	❌
POST	`/api/auth/verify-otp`	Verify OTP code	❌
POST	`/api/auth/reset-password`	Reset password with OTP	❌

Key Files:

- `AuthController.cs` - API endpoints
- `AuthService.cs` - Business logic
- `JwtTokenGenerator.cs` - Token creation
- DTOs: `LoginDto`, `RegisterDto`, `ForgotPasswordDto`, `VerifyOtpDto`, `ResetPasswordDto`

Features:

- ✅ Email/password registration with validation
- ✅ Secure login with password hashing
- ✅ OTP-based password recovery
- ✅ JWT token generation with claims
- ✅ Email notification on registration

5.2 Products Module

Endpoints:

Method	Route	Description	Auth
GET	`/api/products`	Get products with filtering/pagination	❌
GET	`/api/products/{id}`	Get product details	❌
POST	`/api/products`	Create product (Vendor)	✅
PUT	`/api/products/{id}`	Update product (Vendor)	✅
DELETE	`/api/products/{id}`	Delete product (Vendor)	✅

Service Methods:

C#

```
public interface IProductService
{
    Task<PaginatedResult<ProductDto>> GetProductsAsync(ProductFilterDto
filter);
    Task<ProductDetailsDto> GetProductDetailsAsync(int id);
    Task<ProductResponseDto> CreateProductAsync(int workshopId,
CreateProductDto dto);
    Task<ProductResponseDto> UpdateProductAsync(int productId, int
workshopId, UpdateProductDto dto);
    Task<bool> DeleteProductAsync(int productId, int workshopId);
}
```

Filtering & Pagination:

C#

```
[HttpGet]
public async Task<IActionResult> GetProducts(
```

```
    [FromQuery] string search = null,
    [FromQuery] int? categoryId = null,
    [FromQuery] decimal? minPrice = null,
    [FromQuery] decimal? maxPrice = null,
    [FromQuery] string material = null,
    [FromQuery] int? workshopId = null,
    [FromQuery] int pageNumber = 1,
    [FromQuery] int pageSize = 10)
{
    var filter = new ProductFilterDto
    {
        Search = search,
        CategoryId = categoryId,
        MinPrice = minPrice,
        MaxPrice = maxPrice,
        Material = material,
        WorkshopId = workshopId,
        PageNumber = pageNumber,
        PageSize = pageSize
    };

    var result = await _productService.GetProductsAsync(filter);
    return Ok(result);
}
```

5.3 Categories Module

Endpoints:

Method	Route	Description	Auth
GET	`/api/categories`	Get all categories	❌
POST	`/api/categories`	Create category (Admin)	✅ Admin
PUT	`/api/categories/{id}`	Update category (Admin)	✅ Admin
DELETE	`/api/categories/{id}`	Delete category (Admin)	✅ Admin

Service Methods:

C#

```
public interface ICategoryService
{
    Task<List<CategoryResponseDto>> GetAllCategoriesAsync();
    Task<CategoryResponseDto> CreateCategoryAsync(CreateCategoryDto dto);
    Task<CategoryResponseDto> UpdateCategoryAsync(int id, UpdateCategoryDto
dto);
    Task<bool> DeleteCategoryAsync(int id);
}
```

5.4 Cart Module

Endpoints:

Method	Route	Description	Auth
GET	`/api/cart`	Get user's cart	✅
POST	`/api/cart/items`	Add item to cart	✅
PUT	`/api/cart/items`	Update cart item quantity	✅
DELETE	`/api/cart/items/{id}`	Remove item from cart	✅
DELETE	`/api/cart`	Clear entire cart	✅

Service Methods:

C#

```
public interface ICartService
{
    Task<CartDto> GetCartAsync(string userId);
    Task<CartItemDto> AddToCartAsync(string userId, AddToCartDto dto);
    Task UpdateCartItemAsync(string userId, UpdateCartItemDto dto);
    Task RemoveFromCartAsync(string userId, int cartItemId);
    Task ClearCartAsync(string userId);
}
```

DTOs:

- `CartDto` - Complete cart with items and totals
- `CartItemDto` - Individual cart line item
- `AddToCartDto` - New item addition

- `UpdateCartItemDto` - Quantity update

5.5 Orders Module

Endpoints:

Method	Route	Description	Auth
POST	`/api/orders`	Create new order	✅
GET	`/api/orders`	Get all orders (Admin)	✅ Admin
GET	`/api/orders/{id}`	Get order details	✅
GET	`/api/orders/my-orders`	Get user's orders	✅
PUT	`/api/orders/{id}/status`	Update order status (Admin)	✅ Admin
PUT	`/api/orders/{id}/cancel`	Cancel order	✅

Service Methods:

C#

```
public interface IOrderService
{
    Task<OrderResponseDto> CreateOrderAsync(string userId, CreateOrderDto
request, ClaimsPrincipal user);
    Task<OrderResponseDto> GetOrderByIdAsync(int orderId);
    Task<List<OrderResponseDto>> GetMyOrdersAsync(string userId);
    Task UpdateOrderStatusAsync(int orderId, string status);
    Task CancelOrderAsync(int orderId, string userId);
    Task<List<OrderResponseDto>> GetAllOrdersAsync();
}
```

Order Lifecycle:

Code

```
New Order (Cart → Order)
      |
      ▼
Create Order Items (from Cart)
      |
      ▼
```

```
Clear User Cart
      |
      ▼
Send Confirmation Notification (Email + WhatsApp)
      |
      ▼
[Pending] ──(Update)──> [Processing] ──(Update)──> [Shipped] ──(Update)──>
[Delivered]
      |
      └──────(Cancel)──> [Cancelled]
```

5.6 Reviews & Ratings Module

Endpoints:

Method	Route	Description	Auth
GET	`/api/reviews/product/{id}`	Get product reviews	❌
GET	`/api/reviews/product/{id}/rating`	Get average rating	❌
POST	`/api/reviews`	Create review (User)	✅
DELETE	`/api/reviews/{id}`	Delete review (Owner/Admin)	✅
GET	`/api/reviews/{id}`	Get review details	❌

Service Methods:

C#

```
public interface IReviewService
{
    Task<ReviewDto> CreateReviewAsync(string userId, CreateReviewDto dto);
    Task<IEnumerable<ReviewDto>> GetProductReviewsAsync(int productId);
    Task<AverageRatingDto> GetProductAverageRatingAsync(int productId);
    Task<bool> DeleteReviewAsync(int reviewId, string userId);
    Task<ReviewDetailsDto> GetReviewDetailsAsync(int reviewId);
}
```

DTOs:

C#

```
public class CreateReviewDto
{
    public int ProductId { get; set; }
    [Range(1, 5)]
    public int Rating { get; set; }
    public string Comment { get; set; }
}

public class AverageRatingDto
{
    public int ProductId { get; set; }
    public decimal AverageRating { get; set; }
    public int TotalReviews { get; set; }
}
```

5.7 Favorites Module

Endpoints:

Method	Route	Description	Auth
GET	`/api/favorites`	Get user's favorites	✅
POST	`/api/favorites/{productId}`	Add to favorites	✅
DELETE	`/api/favorites/{productId}`	Remove from favorites	✅

Service Methods:

C#

```
public interface IFavoriteService
{
    Task<string> AddToFavoritesAsync(string userId, int productId);
    Task<string> RemoveFromFavoritesAsync(string userId, int productId);
    Task<List<FavoriteDto>> GetUserFavoritesAsync(string userId);
}
```

5.8 Profile Module

Endpoints:

Method	Route	Description	Auth
GET	`/api/profile`	Get user profile	✅
PUT	`/api/profile`	Update profile	✅
PUT	`/api/profile/change-password`	Change password	✅

Service Methods:

C#

```
public interface IProfileService
{
    Task<UserProfileDto> GetProfileAsync(string userId);
    Task<UserProfileDto> UpdateProfileAsync(string userId, UpdateProfileDto
dto);
    Task ChangePasswordAsync(string userId, ChangePasswordDto dto);
}
```

5.9 Notifications Module

Purpose: Multi-channel notification coordination for order updates and transactional alerts.

Service Methods:

C#

```
public interface INotificationService
{
    Task SendNotificationAsync(string userId, string message);
    Task SendOrderConfirmationAsync(string userId, int orderId);
    Task SendOrderStatusUpdateAsync(string userId, int orderId, string
newStatus);
    Task SendOrderCancellationAsync(string userId, int orderId);
}
```

Notification Channels:

- 📧 Email (SendGrid)
- 📱 WhatsApp API

- 🔔 In-app notifications

---

6. API Design & Response Patterns

6.1 REST Conventions

The API adheres to RESTful principles:

Method	Usage	Status	Example
GET	Retrieve resource	200 OK	`GET /api/products/5`
POST	Create resource	201 Created	`POST /api/orders`
PUT	Full update	200 OK	`PUT /api/products/5`
DELETE	Remove resource	204 No Content	`DELETE /api/cart/items/3`

6.2 Response Format

Success Response:

JSON

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": {
    "id": 1,
    "name": "Product Name",
    "price": 99.99
  }
}
```

Error Response:

JSON

```
{
  "success": false,
  "message": "Invalid product ID",
  "errors": [
    {
      "field": "productId",
```

```
      "message": "Product ID must be greater than 0"
    }
  ]
}
```

Paginated Response:

JSON

```
{
  "success": true,
  "data": [
    { "id": 1, "name": "Product 1" },
    { "id": 2, "name": "Product 2" }
  ],
  "pageNumber": 1,
  "pageSize": 10,
  "totalRecords": 50,
  "totalPages": 5
}
```

6.3 HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful GET, PUT, PATCH
201	Created	Successful POST (resource created)
204	No Content	Successful DELETE
400	Bad Request	Validation errors, malformed request
401	Unauthorized	Missing/invalid JWT token
403	Forbidden	Authenticated but insufficient permissions
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource, business logic conflict
500	Internal Server Error	Unhandled exception

---

7. Security Architecture

7.1 Authentication & JWT

JWT Configuration:

C#

```
// JwtTokenGenerator Implementation
public string GenerateToken(ApplicationUser user, IList<string> roles)
{
    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id),
        new Claim(ClaimTypes.Email, user.Email),
        new Claim(ClaimTypes.Name, user.FullName)
    };

    // Add roles as claims
    foreach (var role in roles)
    {
        claims.Add(new Claim(ClaimTypes.Role, role));
    }

    var key = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));
    var credentials = new SigningCredentials(key,
SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _configuration["Jwt:Issuer"],
        audience: _configuration["Jwt:Audience"],
        claims: claims,
        expires: DateTime.UtcNow.AddHours(1),
        signingCredentials: credentials
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

JWT Configuration (appsettings.json):

JSON

```
{
  "Jwt": {
    "Key": "your-256-bit-secret-key-here",
```

```
    "Issuer": "FurniMindAI",
    "Audience": "FurniMindAIUsers",
    "ExpiryMinutes": 60
  }
}
```

7.2 Authorization Strategy

Role-Based Access Control (RBAC):

C#

```
// Admin-only operations
[Authorize(Roles = "Admin")]
[HttpPut("{id}")]
public async Task<IActionResult> UpdateCategory(int id, UpdateCategoryDto
dto)
{
    // Only admins can access
}

// Vendor operations
[Authorize(Roles = "Vendor")]
[HttpPost]
public async Task<IActionResult> CreateProduct(CreateProductDto dto)
{
    // Only vendors can create products
}

// Authenticated users
[Authorize]
[HttpGet]
public async Task<IActionResult> GetCart()
{
    // Any authenticated user
}
```

Available Roles:

- `Admin` - Full system access, category/role management
- `Vendor` - Product creation and management
- `Customer` - Browse, purchase, review products
- `User` - Generic authenticated user role

7.3 Data Security

Password Security:

C#

```
// ASP.NET Core Identity handles password hashing
// Automatically uses PBKDF2 with 10,000 iterations
var passwordHasher = new PasswordHasher<ApplicationUser>();
var hashed = passwordHasher.HashPassword(user, plainTextPassword);
```

Sensitive Data:

- Never log passwords or tokens
- JWT tokens expire after configurable duration (default: 1 hour)
- OTP codes expire after 15 minutes
- Secure cookie settings: `HttpOnly`, `SameSite=Strict`

7.4 CORS Security

C#

```
// Configure CORS with origins
services.AddCors(options =>
{
    options.AddPolicy("AllowAll",
        builder =>
        {
            builder
                .AllowAnyOrigin()              // Configure based on
environment
                .AllowAnyMethod()
                .AllowAnyHeader();
        });
});
```

Production Recommendation:

C#

```
// Use specific origins in production
builder
    .WithOrigins("https://furnimind.ai", "https://www.furnimind.ai")
    .AllowAnyMethod()
```

```
    .AllowAnyHeader()
    .AllowCredentials();
```

---

8. Performance & Scalability

8.1 Async/Await Programming

All I/O operations use asynchronous patterns:

C#

```
// ✅ Good: Async methods
public async Task<ProductDetailsDto> GetProductDetailsAsync(int id)
{
    var product = await _productRepository
        .Include(p => p.Images)
        .FirstOrDefaultAsync(p => p.Id == id);

    return MapToDetailsDto(product);
}

// ❌ Avoid: Synchronous blocking
public ProductDetailsDto GetProductDetails(int id)
{
    var product = _productRepository.FirstOrDefault(p => p.Id == id);
    return MapToDetailsDto(product);
}
```

8.2 Pagination Implementation

C#

```
public class PaginatedResult<T>
{
    public IEnumerable<T> Items { get; set; }
    public int PageNumber { get; set; }
    public int PageSize { get; set; }
    public int TotalRecords { get; set; }
    public int TotalPages => (int)Math.Ceiling((double)TotalRecords /
PageSize);
}
```

```
// Usage in service
public async Task<PaginatedResult<ProductDto>>
GetProductsAsync(ProductFilterDto filter)
{
    var query = _productRepository.GetQueryable();

    // Apply filters...
    if (!string.IsNullOrEmpty(filter.Search))
        query = query.Where(p => p.NameEn.Contains(filter.Search)
                              || p.NameAr.Contains(filter.Search));

    var totalRecords = await query.CountAsync();

    var items = await query
        .Skip((filter.PageNumber - 1) * filter.PageSize)
        .Take(filter.PageSize)
        .ToListAsync();

    return new PaginatedResult<ProductDto>
    {
        Items = items.Select(MapToDto),
        PageNumber = filter.PageNumber,
        PageSize = filter.PageSize,
        TotalRecords = totalRecords
    };
}
```

8.3 Query Optimization

Using Include for Eager Loading:

C#

```
// ✅ Good: Single query with eager loading
var order = await _orderRepository
    .Include(o => o.Items)
        .ThenInclude(oi => oi.Product)
    .Include(o => o.StatusHistory)
    .FirstOrDefaultAsync(o => o.Id == orderId);

// ❌ Avoid: N+1 queries (lazy loading each item)
var order = await _orderRepository.FirstOrDefaultAsync(o => o.Id ==
```

```
orderId);
foreach (var item in order.Items)
{
    var product = await _productRepository.FirstOrDefaultAsync(p => p.Id ==
item.ProductId);
}
```

Projections for Performance:

C#

```
// ✅ Select only needed fields
var products = await query
    .Select(p => new ProductDto
    {
        Id = p.Id,
        Name = p.NameEn,
        Price = p.Price
    })
    .ToListAsync();
```

8.4 Database Connection Pooling

Connection pooling is enabled by default in Entity Framework Core with optimal settings:

Code

```
Default pool size: 10 minimum connections
Maximum pool size: 100 connections
Connection timeout: 15 seconds
```

8.5 Caching Strategy (Future Enhancement)

C#

```
// Recommended for frequently accessed data
services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = configuration.GetConnectionString("Redis");
});

// Cache product categories (rarely changing)
var categories = await _cache.GetOrCreateAsync("categories:all", async entry
```

```
=>
{
    entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(1);
    return await _categoryService.GetAllCategoriesAsync();
});
```

---

9. Environment Configuration

9.1 appsettings Configuration

Development (appsettings.Development.json):

JSON

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Debug",
      "Microsoft": "Debug"
    }
  },
  "ConnectionStrings": {
    "GraduationDbOnline": "Server=localhost;Database=GraduationDb;..."
  },
  "Jwt": {
    "Key": "dev-secret-key-256-bits-minimum",
    "ExpiryMinutes": 60
  },
  "EmailService": {
    "ApiKey": "dev-sendgrid-key"
  },
  "WhatsAppNotification": {
    "DefaultTemplateName": "order_confirmation",
    "DefaultLanguageCode": "ar"
  }
}
```

Production (appsettings.Production.json):

JSON

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Error"
    }
  },
  "ConnectionStrings": {
    "GraduationDbOnline": "Server=prod-db-server;Database=GraduationDb;..."
  },
  "Jwt": {
    "Key": "prod-secret-key-from-keyvault",
    "ExpiryMinutes": 30
  }
}
```

9.2 Environment Variables

Sensitive configurations should use environment variables or Azure Key Vault:

bash

```
# Database
export ConnectionStrings__GraduationDbOnline="Server=...;Password=...;"

# JWT
export Jwt__Key="your-secret-key-here"
export Jwt__Issuer="FurniMindAI"

# Email Service
export EmailService__ApiKey="sendgrid-api-key"

# WhatsApp
export WhatsAppNotification__ApiKey="whatsapp-api-key"
```

9.3 Configuration Retrieval

C#

```
// In services or controllers
var connectionString =
_configuration.GetConnectionString("GraduationDbOnline");
```

```
var jwtKey = _configuration["Jwt:Key"];
var emailApiKey = _configuration["EmailService:ApiKey"];
```

---

10. Setup & Installation

10.1 Prerequisites

- .NET 8.0 SDK or higher
- SQL Server 2019 or higher (or SQL Server Express)
- Git
- Visual Studio 2022 (recommended) or VS Code

10.2 Clone Repository

bash

```
git clone https://github.com/Ahmed-Adam0/HomeAi.git
cd HomeAi
```

10.3 Restore Dependencies

bash

```
# Restore NuGet packages for all projects
dotnet restore
```

10.4 Configure Database Connection

Edit `Graduation-API/appsettings.json`:

JSON

```
{
  "ConnectionStrings": {
    "GraduationDbOnline": "Server=YOUR_SERVER;Database=GraduationDb;User
Id=YOUR_USER;Password=YOUR_PASSWORD;Encrypt=True;TrustServerCertificate=True
;"
  }
}
```

Or set environment variable:

bash

```
export ConnectionStrings__GraduationDbOnline="Server=...;User
Id=...;Password=...;"
```

10.5 Apply Database Migrations

bash

```
# Navigate to API project
cd Graduation-API

# Apply pending migrations
dotnet ef database update

# If migrations don't exist, create initial migration
dotnet ef migrations add InitialCreate
dotnet ef database update
```

10.6 Run the Application

bash

```
cd Graduation-API

# Development mode
dotnet run

# Production mode
dotnet run --configuration Release
```

API will be available at: `http://localhost:5000` or `https://localhost:5001`

Swagger/OpenAPI documentation: `http://localhost:5000/swagger`

10.7 Database Seeding

On application startup, the `ApplicationDbSeeder` automatically:

- Creates default admin role
- Creates default admin user
- Seeds initial categories
- Seeds sample products

11. API Endpoints Overview

Authentication Endpoints

Endpoint	Method	Body	Response
`/api/auth/register`	POST	`{email, password, fullName}`	`{token, user, roles}`
`/api/auth/login`	POST	`{email, password}`	`{token, user, roles}`
`/api/auth/forgot-password`	POST	`{email}`	`{message: "OTP sent"}`
`/api/auth/verify-otp`	POST	`{email, otp}`	`{isValid: true}`
`/api/auth/reset-password`	POST	`{email, otp, newPassword}`	`{message: "Reset successful"}`

Products Endpoints

Endpoint	Method	Purpose	Auth
`/api/products`	GET	List products (paginated, filtered)	❌
`/api/products/{id}`	GET	Get product details	❌
`/api/products`	POST	Create product (vendor)	✅
`/api/products/{id}`	PUT	Update product (vendor)	✅
`/api/products/{id}`	DELETE	Delete product (vendor)	✅

Cart Endpoints

Endpoint	Method	Purpose	Auth
`/api/cart`	GET	Get user cart	✅
`/api/cart/items`	POST	Add to cart	✅
`/api/cart/items`	PUT	Update item quantity	✅
`/api/cart/items/{id}`	DELETE	Remove item	✅
`/api/cart`	DELETE	Clear cart	✅

Orders Endpoints

Endpoint	Method	Purpose	Auth
`/api/orders`	POST	Create order	✅
`/api/orders`	GET	Get all orders (admin)	✅ Admin
`/api/orders/{id}`	GET	Get order details	✅
`/api/orders/my-orders`	GET	Get user orders	✅
`/api/orders/{id}/status`	PUT	Update status (admin)	✅ Admin
`/api/orders/{id}/cancel`	PUT	Cancel order	✅

Reviews Endpoints

Endpoint	Method	Purpose	Auth
`/api/reviews/product/{id}`	GET	Get product reviews	❌
`/api/reviews/product/{id}/rating`	GET	Get avg rating	❌
`/api/reviews`	POST	Create review	✅
`/api/reviews/{id}`	GET	Get review details	❌
`/api/reviews/{id}`	DELETE	Delete review	✅

Favorites Endpoints

Endpoint	Method	Purpose	Auth
`/api/favorites`	GET	Get user favorites	✅
`/api/favorites/{id}`	POST	Add to favorites	✅
`/api/favorites/{id}`	DELETE	Remove from favorites	✅

Categories Endpoints

Endpoint	Method	Purpose	Auth
`/api/categories`	GET	Get all categories	❌
`/api/categories`	POST	Create category (admin)	✅ Admin
`/api/categories/{id}`	PUT	Update category (admin)	✅ Admin
`/api/categories/{id}`	DELETE	Delete category (admin)	✅ Admin

Profile Endpoints

Endpoint	Method	Purpose	Auth
`/api/profile`	GET	Get user profile	✅
`/api/profile`	PUT	Update profile	✅
`/api/profile/change-password`	PUT	Change password	✅

---

12. Team Workflow & Scalability

12.1 Code Organization for Team Collaboration

The solution structure supports parallel team development:

Code

```
Team A: Authentication & Identity
├── AuthController.cs
├── AuthService.cs
├── JwtTokenGenerator.cs
└── Identity/ Configuration

Team B: Products & Categories
├── ProductsController.cs
├── CategoriesController.cs
├── ProductService.cs
├── CategoryService.cs
└── Product/Category DTOs & Entities

Team C: Orders & Payments
├── OrderController.cs
├── OrderService.cs
├── NotificationService.cs
└── Order DTOs & Entities

Team D: User Features (Cart, Favorites, Profile)
├── CartController.cs
├── FavoritesController.cs
├── ProfileController.cs
└── Cart/Favorite/Profile Services
```

12.2 Development Workflow

1. **Feature Branch Workflow**

   bash

   ```
   git checkout -b feature/product-filtering
   git add .
   git commit -m "feat: implement advanced product filtering"
   git push origin feature/product-filtering
   # Create Pull Request
   ```

2. **Code Review Guidelines**
   - Verify async/await usage
   - Check error handling consistency
   - Validate authorization attributes
   - Ensure DTO mapping correctness
3. **Testing Standards**
   - Unit tests for services
   - Integration tests for controllers
   - Database migration testing

12.3 Scalability Considerations

Horizontal Scaling:

- Stateless API design (JWT-based)
- Load balancing ready
- Session-independent architecture

Vertical Scaling:

- Connection pooling optimization
- Query optimization with indexes
- Caching layer (Redis-ready)

Database Scaling:

- Consider read replicas for heavy queries
- Implement database indexing strategy
- Archive old orders/notifications

Microservices Ready:

- Independent service boundaries
- Communication via REST APIs
- Easily isolated into microservices

13. Future Improvements & Roadmap

Phase 1: Performance Enhancements (Q2 2026)

- [ ] Implement Redis caching for categories and products
- [ ] Add database query indexes for frequently accessed tables
- [ ] Implement GraphQL API alongside REST
- [ ] Add request/response compression

Phase 2: Advanced Features (Q3 2026)

- [ ] AI-powered product recommendations
- [ ] Real-time notifications using SignalR
- [ ] Inventory management system
- [ ] Multi-warehouse support
- [ ] Advanced analytics dashboard

Phase 3: Infrastructure & DevOps (Q4 2026)

- [ ] Docker containerization
- [ ] Kubernetes orchestration
- [ ] CI/CD pipeline (GitHub Actions)
- [ ] Azure deployment templates
- [ ] Health check endpoints

Phase 4: Security & Compliance (Q1 2027)

- [ ] OAuth 2.0 integration (Google, Microsoft)
- [ ] Two-factor authentication (2FA)
- [ ] PCI-DSS compliance for payments
- [ ] GDPR data handling policies
- [ ] Rate limiting and DDoS protection

Phase 5: Mobile & Integration (Q2 2027)

- [ ] Mobile API optimization
- [ ] Push notification system
- [ ] SMS payment verification
- [ ] Third-party ERP integration
- [ ] Accounting software integration (QuickBooks)

14. Troubleshooting & Common Issues

Database Connection Issues

Code

```
Error: Cannot open database "GraduationDb"
Solution: Verify connection string and SQL Server is running
Command: sqlcmd -S YOUR_SERVER -U YOUR_USER -P YOUR_PASSWORD
```

JWT Token Expiration

Code

```
Error: 401 Unauthorized - token expired
Solution: Token expires after configured duration (default 1 hour)
Action: Client should refresh token or re-login
```

Migration Errors

Code

```
Error: The model backing the context has changed
Solution: Create and apply new migration
Commands:
  dotnet ef migrations add UpdateDatabase
  dotnet ef database update
```

CORS Errors

Code

```
Error: Access to XMLHttpRequest blocked by CORS policy
Solution: Verify CORS configuration in Program.cs
Ensure frontend origin is allowed in CORS policy
```

---

15. Contributors & Team

Meet the talented team behind **FurniMind AI**:

Name	Role	GitHub	LinkedIn
Ahmed Adam	Backend Lead	@Ahmed-Adam0	Profile
Saleh	Backend Developer	@Saleh-Username	Profile
Sultan	Backend Developer	@Sultan-Username	Profile
Ashraf	Full Stack Developer	@Ashraf-Username	Profile
Mayar	Frontend Developer	@Mayar-Username	Profile
Ayat	Frontend Developer	@Ayat-Username	Profile

---

16. License & Support

This project is part of an educational initiative. For questions or support:

📧 **Email:** support@furnimind.ai 📱 **WhatsApp:** Contact 🌐 **Website:** https://furnimind.ai
📖 **Documentation:** GitHub Wiki

---

17. Additional Resources

Reference Documentation

- ASP.NET Core Official Docs
- Entity Framework Core Docs
- JWT.io Token Debugger
- REST API Best Practices

Related Projects

- **Frontend:** Angular 20 Enterprise Portal
- **Mobile:** React Native Mobile App

Related Documentation

- Frontend README - Angular portal architecture
- Database Schema Diagram
- API Postman Collection

Last Updated: May 2026
Backend Version: 1.0.0
ASP.NET Core: 8.0.x
Entity Framework Core: 10.0.x